NagaEd Software Developer Hurdle Task

Candidate name: **Satyam Singh**
Contact: satyamsingh7734@gmail.com / +91 9918437706

A complete registration and authentication feature with a React frontend, Express/MySQL backend, FastAPI username suggestions using LangChain Groq, and a Docker Compose deployment. See `architecture.png` for services and ports.

For environment setup, builds, local development, tests and troubleshooting, see Developer guide.

Screenshots

Home page	Signed in UI	AI username suggestions
		

Run with Docker

Requires Docker Desktop with Linux containers and Docker Compose v2. From the project root in PowerShell:

```
Copy-Item .env.example .env
# Optional: put your Groq key in GROQ_API_KEY in .env.
docker compose up --build -d
docker compose ps
```

Open http://localhost:8080. AI Swagger: http://localhost:8080/docs. The first startup downloads images and automatically initializes MySQL; allow a minute for the database to become healthy. Only the web port is published. A missing Groq key leaves signup/login working and returns a clear 503 for suggestions. To enable real suggestions, obtain a key from https://console.groq.com, set it in `.env`, then run `docker compose up -d ai`.

Root `.env` drives Compose. Each service also has its own `.env.example` for running outside Docker; those service-specific files are not needed by Compose. The frontend has no secrets. The example database/JWT values are local evaluation values, intentionally documented in `.env.example`; replace them with unique, random values before public deployment. Actual `.env` files are ignored and never included in the submission ZIP.

```
docker compose logs --tail=100 api ai db web
python scripts/smoke-test.py
# If Python is not installed locally:
docker compose exec -T ai python - < scripts/smoke-test.py
```

The last command requires a shell supporting `<` redirection (for example Bash). In PowerShell use `Get-Content -Raw scripts/smoke-test.py | docker compose exec -T ai python - http://web:8080`. The smoke

test creates one uniquely named account; it checks real MySQL registration, duplicate email, login, protected profile, ownership, OpenAPI and invalid AI input. It intentionally does not call paid Groq APIs.

```
# Stop containers, keeping the database:
docker compose down
```

Database data persists in the `mysql_data` named volume. The schema under `backend/database/001-users.sql` runs only when that volume is first initialized. Changing database credentials in `.env` does not update an existing database user; update the MySQL user explicitly if reusing an initialized volume.

Development and tests

See `backend/README.md`, `frontend/README.md` and `ai-service/README.md` for local setup. Use Node 22, Python 3.12, MySQL 8.4. With dependencies installed:

```
cd backend
npm install
npm test
npm run format
npm run format:check
cd ../frontend
npm install
npm run build
npm run format
npm run format:check
cd ../ai-service
python -m pip install -r requirements.txt
python -m unittest discover -s tests -v
cd ..
docker compose --env-file .env.example config --quiet
```

Prettier configuration and scripts are included in both JavaScript services. The first `npm install` generates a lockfile; commit it and use `npm ci` for repeatable CI builds. Lockfiles could not be generated here because package registry access was blocked, so Docker currently uses `npm install`.

Structure and API

```
backend/
  database/001-users.sql
  src/{config,controllers,middleware,repositories,routes,services,utils}
  test/
frontend/
  src/{main.jsx,api.js,style.css}
ai-service/
  app/{api,core,services,providers,prompts,schemas,utils}
  tests/
scripts/smoke-test.py
architecture.png
docker-compose.yml
```

Method	Endpoint	Purpose
POST	/api/register	username, email, password, address -> 201 safe user
POST	/api/login	email, password -> token and userId
GET	/api/users/:id	Bearer JWT; own profile only
POST	/api/suggest-username	interests array -> 3 unique creative names
GET	/docs	FastAPI interactive API documentation

Requests and failures use consistent JSON; error messages are under `error.message`. AI inputs allow 1-10 interests with 1-50 characters each. Usernames are 3-30 ASCII letters/numbers/underscores. Emails are normalized and unique. Passwords require at least 8 characters and at most 72 UTF-8 bytes, are bcrypt-hashed at cost 12, and never appear in responses. JWTs expire after one hour and validate algorithm, issuer and audience. Parameterized SQL prevents injection. AI suggestions are creative names, not reserved or globally unique accounts; duplicate usernames are allowed by the assignment's email identity model.

Deployment boundaries

Included: Helmet, browser security headers/CSP, body-size limits, request throttling (general API, login, registration, AI gateway), trusted proxy boundaries, input and output validation, secret environment variables, non-root containers, health checks, dependency readiness, persistent MySQL storage and graceful Node shutdown. Requests stay same-origin through Nginx, so no broad CORS allowance is needed. Backend authentication logic is tested with an injected repository; the smoke test covers the real database path.

Before public production use, terminate HTTPS at a trusted load balancer, rotate example secrets, configure backups/monitoring, pin image digests and dependency lockfiles, and replace in-memory Node rate limits with a shared store when scaling. JWT logout clears the browser token but does not revoke an already copied token. This task does not include password reset, email verification, refresh tokens or account recovery. The browser token is tab-scoped sessionStorage; prevent XSS and consider an HttpOnly cookie/session design for a larger product.

Verification and submission

Checked here: AI unit tests (10 passing), Python lint/format, standalone backend validation tests (3 passing), JavaScript syntax where supported by Node, and Compose configuration. Complete Node tests, Prettier execution and live Groq calls still need verification where Docker/package downloads are permitted. The supplied Docker logs confirm the React production build succeeded and the application containers started, with MySQL, Express and FastAPI healthy. End-to-end smoke/browser verification remains pending. A live AI call requires your own Groq key and model access.

Before submitting, run the tests and browser flows, and recreate the ZIP after any changes:

```
python scripts/package-submission.py
```

Upload `nagaed-submission.zip` (or a public GitHub repository) using the form in the assignment. Use subject `Satyam Singh - Software Developer Hurdle Task Submission`. The form has not been submitted on your behalf.

Integration references: LangChain ChatGroq, Groq JSON mode.